



**Universidad Privada del Valle
Facultad de Informática y Electrónica
Carrera de Licenciatura en Ingeniería de Sistemas**

**MÓDULO DE INTELIGENCIA PREDICTIVA DEL ERP ISI MUSTANG
BASADO EN EL APROVECHAMIENTO DE DATOS HISTÓRICOS
OPERATIVOS PARA ANTICIPAR DESVIACIONES Y RIESGOS EN
LA GESTIÓN DE PROYECTOS DE INGENIERÍA EPC**

**PROYECTO DE GRADO PARA OPTAR
AL TÍTULO DE
LICENCIATURA EN INGENIERÍA DE SISTEMAS**

**POSTULANTE: FRANCO JAVIER TORREZ ALVARADO
TUTOR: ROLANDO LARA SANCHEZ**

Santa Cruz – Bolivia

2026

Índice

INTRODUCCIÓN	1
CAPÍTULO I: MARCO GENERAL	2
1.1 ANTECEDENTES	3
1.2 PLANTEAMIENTO DEL PROBLEMA	3
1.2.1 FORMULACIÓN DEL PROBLEMA	3
1.3 OBJETIVOS	3
1.3.1 OBJETIVO GENERAL	3
1.3.2 OBJETIVOS ESPECÍFICOS	3
1.4 JUSTIFICACIÓN	3
1.4.1 JUSTIFICACIÓN TÉCNICA	3
1.4.2 JUSTIFICACIÓN ECONÓMICA	3
1.4.3 JUSTIFICACIÓN SOCIAL	3
1.5 METODOLOGÍA	4
1.5.1 TIPO DE INVESTIGACIÓN	4
1.5.2 MÉTODO DE INVESTIGACIÓN	4
1.5.3 TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN	4
1.5.4 POBLACIÓN Y MUESTRA	4
CAPÍTULO II: MARCO TEÓRICO Y TECNOLÓGICO	5
2.1 SISTEMAS ERP	6
2.1.1 DEFINICIÓN Y CARACTERÍSTICAS	6
2.1.2 ERP EN EMPRESAS DE INGENIERÍA	6
2.1.3 ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA	6
2.2 APRENDIZAJE AUTOMÁTICO	6
2.2.1 TIPOS DE APRENDIZAJE	6
2.2.2 APRENDIZAJE SUPERVISADO	7
2.2.3 ALGORITMOS DE APRENDIZAJE SUPERVISADO: GRADIENT BOOSTING Y XGBOOST	10
2.2.4 FEATURE ENGINEERING	12
2.3 INGENIERÍA DE DATOS	13
2.3.1 PROCESOS ETL Y ELT	13
2.3.2 ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD)	13
2.3.3 CALIDAD DE DATOS	13
2.4 ANALÍTICA PREDICTIVA	13
2.4.1 DEFINICIÓN Y APLICACIONES	13
2.4.2 ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS	13
2.4.3 PREDICCIÓN DE DESVIACIONES Y COSTOS	14
2.5 GESTIÓN DE PROYECTOS EPC	14
2.5.1 DEFINICIÓN Y CARACTERÍSTICAS	14

2.5.2	CERTIFICACIONES Y CONTROL DE AVANCE	14
2.5.3	DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA	14
2.6	INGENIERÍA DE SOFTWARE	14
2.6.1	PROCESO DE DESARROLLO DE SOFTWARE	14
2.6.2	METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL	14
2.6.3	ARQUITECTURA DE SOFTWARE POR CAPAS	14
2.7	TECNOLOGÍAS DEL STACK DE DESARROLLO	14
2.7.1	PYTHON Y SU ECOSISTEMA ML	14
2.7.2	DUCKDB Y POLARS	15
2.7.3	FASTAPI COMO MICROSERVICIO	15
2.7.4	NESTJS Y ANGULAR	15
2.8	GESTIÓN DE IDENTIDAD Y ACCESO	15
2.8.1	OAuth 2.0 Y OPENID CONNECT	15
2.8.2	KEYCLOAK	15
2.9	MODEL CONTEXT PROTOCOL (MCP)	15
2.9.1	DEFINICIÓN Y ARQUITECTURA	15
2.9.2	INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE	15
2.10	INTELIGENCIA DE NEGOCIOS	15
2.10.1	DEFINICIÓN Y REPORTES DESCRIPTIVOS	15
2.10.2	POWER BI	16
CAPÍTULO III: ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO		17
3.1	ANÁLISIS DE DATOS HISTÓRICOS	18
3.1.1	EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS	18
3.1.2	IDENTIFICACIÓN DE VARIABLES PREDICTORAS	18
3.1.3	DEFINICIÓN DE CASOS DE USO PREDICTIVOS	18
3.2	ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR	18
3.2.1	RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR	18
3.2.2	PROCESOS A REIMPLEMENTAR EN EL ERP	18
3.2.3	DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS	18
3.3	REQUERIMIENTOS DEL SISTEMA	18
3.3.1	REQUERIMIENTOS FUNCIONALES	18
3.3.2	REQUERIMIENTOS NO FUNCIONALES	18
3.4	DISEÑO DE LA ARQUITECTURA	19
3.4.1	ARQUITECTURA POR CAPAS	19
3.4.2	DISEÑO DEL PIPELINE DE DATOS	19
3.4.3	DISEÑO DE MODELOS ML	19
3.4.4	DISEÑO DE LA API PREDICTIVA	19
3.4.5	DISEÑO DEL DASHBOARD	19
3.4.6	DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA	19
3.4.7	DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI	19

CAPÍTULO IV: CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA	20
4.1 INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP . . .	21
4.2 INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK	21
4.3 INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS . . .	21
4.3.1 INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL	21
4.3.2 TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING	21
4.3.3 MODELO DE DESVIACIÓN EN CERTIFICACIONES	21
4.3.4 MODELO DE SOBRECOSTO DE PROYECTOS	21
4.3.5 MODELO DE RETRASO EN LA FECHA DE CIERRE	21
4.4 INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP . . .	21
4.4.1 DESARROLLO DEL MICROSERVICIO FASTAPI	21
4.4.2 ENDPOINTS DE PREDICCIÓN EN NESTJS	21
4.4.3 DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR	22
4.5 INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP	22
4.5.1 API DE DATOS PARA POWER BI	22
4.5.2 SERVIDOR MCP	22
4.6 EVALUACIÓN Y VALIDACIÓN	22
4.6.1 MÉTRICAS DE RENDIMIENTO DE MODELOS	22
4.6.2 PRUEBAS CON USUARIOS CLAVE	22
CONCLUSIONES	23
RECOMENDACIONES	24
REFERENCIAS BIBLIOGRÁFICAS	25

ÍNDICE DE FIGURAS

ÍNDICE DE CUADROS

2.1 Métricas de evaluación de modelos supervisados y su uso en el proyecto	9
--	---

INTRODUCCIÓN

CAPÍTULO I

MARCO GENERAL

1.1 ANTECEDENTES

1.2 PLANTEAMIENTO DEL PROBLEMA

1.2.1 FORMULACIÓN DEL PROBLEMA

1.3 OBJETIVOS

1.3.1 OBJETIVO GENERAL

1.3.2 OBJETIVOS ESPECÍFICOS

1.4 JUSTIFICACIÓN

1.4.1 JUSTIFICACIÓN TÉCNICA

1.4.2 JUSTIFICACIÓN ECONÓMICA

1.4.3 JUSTIFICACIÓN SOCIAL

1.5 METODOLOGÍA

1.5.1 TIPO DE INVESTIGACIÓN

1.5.2 MÉTODO DE INVESTIGACIÓN

1.5.3 TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN

1.5.4 POBLACIÓN Y MUESTRA

CAPÍTULO II

MARCO TEÓRICO Y TECNOLÓGICO

2.1 SISTEMAS ERP

2.1.1 DEFINICIÓN Y CARACTERÍSTICAS

2.1.2 ERP EN EMPRESAS DE INGENIERÍA

2.1.3 ERP EN EL SECTOR DE PETRÓLEO, GAS Y MINERÍA

2.2 APRENDIZAJE AUTOMÁTICO

2.2.1 TIPOS DE APRENDIZAJE

El aprendizaje automático (*machine learning*) es la rama de la inteligencia artificial que estudia los algoritmos capaces de mejorar su desempeño de manera automática a partir de la experiencia. **Mitchell1997<empty citation>** lo formaliza con tres elementos: se dice que un programa aprende de una experiencia E respecto de una clase de tareas T y una medida de desempeño P si su desempeño en T , medido por P , mejora con E . Trasladada al presente proyecto, la experiencia es el histórico operativo de más de diez años de ISI Mustang, la tarea es anticipar desviaciones en sus proyectos y la medida de desempeño son las métricas que se describen en la subsección siguiente.

La forma en que esa experiencia se presenta al algoritmo define los paradigmas clásicos del campo. **Geron2022<empty citation>** organiza los sistemas de aprendizaje automático según la supervisión que reciben durante el entrenamiento en tres familias: supervisado, no supervisado y por refuerzo. En el aprendizaje supervisado cada observación de entrenamiento viene acompañada de la respuesta correcta, la etiqueta o valor objetivo, y el algoritmo aprende una función que asocia las entradas con esa salida para predecirla sobre observaciones nuevas

(Hastie2009). En el aprendizaje no supervisado no existe tal respuesta: el algoritmo recibe únicamente las entradas y busca estructura en ellas, como grupos de observaciones similares o direcciones de mayor variabilidad (James2021). El aprendizaje por refuerzo, por último, no parte de un conjunto de datos fijo sino de un agente que interactúa con un entorno y ajusta su comportamiento según las recompensas que recibe (Mitchell1997).

Dentro del aprendizaje supervisado, la naturaleza de la variable de salida distingue dos tipos de problema. Cuando la salida es cuantitativa (un monto, una duración, un porcentaje) se habla de regresión; cuando es cualitativa o categórica (una clase entre un conjunto finito) se habla de clasificación (Hastie2009; James2021). La distinción no es solo nominal: determina qué algoritmos son aplicables, qué función de pérdida se optimiza y con qué métricas se evalúa el resultado.

Los tres casos de uso del módulo predictivo se ubican en el paradigma supervisado, porque el histórico de la empresa ya contiene el resultado de cada proyecto cerrado: se conoce cuánto costó respecto de lo presupuestado, cuándo se cerró respecto de la fecha comprometida y cómo evolucionaron sus certificaciones frente a lo planificado. La predicción de desviación en certificaciones es un problema de clasificación, mientras que la estimación del sobrecosto y del retraso en la fecha de cierre son problemas de regresión. El aprendizaje no supervisado y el aprendizaje por refuerzo quedan fuera del alcance del proyecto.

2.2.2 APRENDIZAJE SUPERVISADO

Formalmente, el aprendizaje supervisado parte de un conjunto de entrenamiento compuesto por pares (x_i, y_i) , donde x_i es un vector de variables predictoras y y_i la respuesta observada, y busca una función f tal que $f(x)$ aproxime a y con el menor error posible sobre observaciones que no participaron del entrenamiento (Hastie2009). Esta última condición es la que separa aprender de memorizar. James2021<empty citation> distinguen el error de entrenamiento, que mide el ajuste sobre los datos vistos, del error de prueba, que mide la capacidad de generalizar; un modelo puede reducir el primero indefinidamente aumentando su complejidad, pero a partir de cierto punto el segundo empieza a crecer. Ese fenómeno es el sobreajuste, y se explica por el compromiso entre sesgo y varianza: los modelos simples tienen sesgo alto porque no capturan la estructura de los

datos, mientras que los modelos muy flexibles tienen varianza alta porque se ajustan al ruido de la muestra particular con la que fueron entrenados (**Hastie2009**; **James2021**).

Como el error de prueba no puede calcularse sobre los datos de entrenamiento, se requiere un procedimiento para estimarlo. La práctica básica consiste en separar una parte de los datos como conjunto de prueba, que se reserva y solo se usa al final para evaluar el modelo elegido (**Geron2022**). Para seleccionar entre modelos o ajustar hiperparámetros sin contaminar ese conjunto se emplea la validación cruzada de k particiones (*k-fold*): los datos de entrenamiento se dividen en k bloques, el modelo se entrena k veces dejando fuera un bloque distinto en cada corrida y el error se promedia sobre las k evaluaciones (**James2021**). En un estudio empírico de referencia, **Kohavi1995**^{<empty citation>} concluye que la validación cruzada estratificada de diez particiones es el mejor método para selección de modelos, incluso cuando la capacidad de cómputo permite usar más particiones, y advierte que la variante *leave-one-out*, aunque casi insesgada, tiene alta varianza y produce estimaciones poco confiables. La estratificación conserva en cada bloque la proporción de clases del conjunto completo, lo que importa cuando una de las clases es minoritaria.

La evaluación del modelo depende del tipo de problema. En clasificación, las medidas usuales derivan de la matriz de confusión, que cruza la clase real con la clase predicha y separa los verdaderos positivos (VP), verdaderos negativos (VN), falsos positivos (FP) y falsos negativos (FN) (**Sokolova2009**; **Geron2022**). A partir de ella se calculan la exactitud (proporción de aciertos sobre el total), la precisión (proporción de positivos predichos que realmente lo son), la sensibilidad o *recall* (proporción de positivos reales que el modelo detecta) y la medida F1, media armónica de las dos anteriores. **Sokolova2009**^{<empty citation>} muestran que estas medidas no reaccionan igual ante cambios en la distribución de clases: la exactitud puede ser alta en un conjunto desbalanceado aunque el modelo ignore la clase minoritaria, razón por la cual conviene reportarlas juntas. En regresión, el punto de partida es el error cuadrático medio (MSE), promedio de las diferencias al cuadrado entre valor observado y predicho (**James2021**); su raíz, el RMSE, se expresa en las unidades de la variable y penaliza con más fuerza los errores grandes, mientras que el error absoluto medio (MAE) promedia las diferencias en valor absoluto y es menos sensible a valores extremos. **Chai2014**^{<empty citation>} sostienen que el RMSE es más apropiado cuando se espera que los errores sigan una distribución gaussiana y que, en general, evaluar un modelo requiere una combinación de métricas y no una sola. A estas se suma el coeficiente de

determinación R^2 , que expresa la proporción de la variabilidad de la respuesta que el modelo explica y permite comparar el ajuste entre variables de distinta escala. El cuadro 2.1 resume las métricas y su uso en el proyecto.

Cuadro 2.1. Métricas de evaluación de modelos supervisados y su uso en el proyecto

Métrica	Qué mide	Uso en el proyecto
<i>Clasificación</i>		
Matriz de confusión	Cruce de clase real y clase predicha (VP, VN, FP, FN)	Base de las demás métricas; muestra qué tipo de error comete el clasificador de desviación en certificaciones
Exactitud	$(VP + VN)/total$	Referencia general; insuficiente si las clases están desbalanceadas
Precisión	$VP/(VP + FP)$	Proporción de alertas de desviación que son correctas
Sensibilidad	$VP/(VP + FN)$	Proporción de desviaciones reales que el modelo detecta
F1	Media armónica de precisión y sensibilidad	Resumen único cuando ambas importan
<i>Regresión</i>		
RMSE	Raíz del promedio de errores al cuadrado	Error en unidades de la variable; penaliza sobrecostos o retrasos grandes mal estimados
MAE	Promedio de errores en valor absoluto	Error típico esperado, robusto a valores extremos
R^2	Proporción de la variabilidad explicada por el modelo	Comparación del ajuste entre el modelo de sobrecosto y el de retraso

Fuente: Elaboración propia.

Estas definiciones fijan el protocolo de evaluación del módulo predictivo: el clasificador de desviación en certificaciones se evaluará con exactitud, precisión, sensibilidad, F1 y matriz de confusión; los regresores de sobrecosto y de retraso en la fecha de cierre, con RMSE, MAE y R^2 ;

y en los tres casos la selección de hiperparámetros se hará con validación cruzada estratificada sobre el conjunto de entrenamiento, reservando un conjunto de prueba independiente para la evaluación final.

2.2.3 ALGORITMOS DE APRENDIZAJE SUPERVISADO: GRADIENT BOOSTING Y XGBOOST

El aprendizaje base de los algoritmos que usa este proyecto es el árbol de decisión. Un árbol particiona recursivamente el espacio de las variables predictoras mediante reglas binarias del tipo «presupuesto mayor que cierto umbral» y asigna a cada región resultante una predicción constante: la clase mayoritaria en clasificación o el promedio de las respuestas en regresión (**Hastie2009**; **James2021**). El procedimiento CART formalizado por **Breiman1984**<empty citation> es el que implementan Scikit-learn y las bibliotecas de *boosting*. Los árboles son fáciles de interpretar y admiten sin transformación previa variables numéricas y categóricas, pero un árbol aislado tiene alta varianza (pequeños cambios en los datos producen árboles muy distintos) y su capacidad predictiva suele ser inferior a la de otros métodos (**James2021**).

Los métodos de ensamble corrigen esa debilidad combinando muchos árboles, con dos estrategias distintas. En el *bagging* y los bosques aleatorios de **Breiman2001**<empty citation>, los árboles se entrenan en paralelo sobre remuestreos de los datos y sus predicciones se promedian, lo que reduce la varianza. En el *boosting*, en cambio, los árboles se construyen de forma secuencial: cada nuevo árbol se ajusta a lo que los anteriores todavía no explican, de modo que el ensamble mejora por etapas (**Hastie2009**; **James2021**). **Friedman2001**<empty citation> dio a esta idea su formulación general al plantear la estimación de la función como un problema de optimización numérica en el espacio de funciones: en cada etapa se ajusta un árbol de regresión al gradiente negativo de la función de pérdida evaluada sobre el modelo acumulado, lo que equivale a un descenso por gradiente en el que cada paso es un árbol. El marco admite distintas funciones de pérdida (mínimos cuadrados, desviación absoluta o Huber para regresión, y verosimilitud logística para clasificación), de modo que el mismo algoritmo sirve para los problemas de regresión y de clasificación del proyecto. Sus hiperparámetros principales son el número de árboles, la tasa de aprendizaje o *shrinkage*, que reduce la contribución de cada árbol, y la profundidad de los

árboles, que controla el orden de las interacciones entre variables (**James2021**); el submuestreo de observaciones en cada etapa agrega una regularización adicional (**Hastie2009**).

XGBoost es la implementación de *gradient boosting* de árboles que utiliza el proyecto. **Chen2016<empty citation>** la presentan como un sistema escalable, ampliamente usado para obtener resultados de vanguardia en competencias de aprendizaje automático, y describen sus aportes: un algoritmo consciente de la dispersión (*sparsity-aware*) para datos dispersos, un esquema de cuantiles ponderados para construir árboles de forma aproximada, y optimizaciones de caché, compresión y particionado que le permiten escalar a miles de millones de ejemplos con menos recursos que otros sistemas. En el diseño del módulo predictivo se prevé aprovechar el tratamiento de valores ausentes que ofrece la biblioteca, dado que el histórico del ERP presenta registros incompletos, y ajustar mediante validación cruzada tanto los hiperparámetros del *boosting* ya mencionados como los parámetros de regularización que XGBoost expone sobre la complejidad de los árboles.

La elección de *gradient boosting* frente a alternativas más recientes se apoya en evidencia sobre datos tabulares. **Grinsztajn2022<empty citation>** compararon modelos de árboles y redes neuronales profundas sobre 45 conjuntos de datos tabulares y concluyeron que los primeros siguen siendo el estado del arte en conjuntos de tamaño medio, del orden de diez mil observaciones, aun sin considerar su ventaja en velocidad; atribuyen el resultado a sesgos inductivos de los árboles, como la robustez frente a variables no informativas y la capacidad de aprender funciones irregulares, que las redes no comparten. **ShwartzZiv2022<empty citation>** llegaron a la misma conclusión al enfrentar XGBoost con modelos profundos diseñados específicamente para datos tabulares: XGBoost los superó incluso en los conjuntos de datos usados por los artículos que los proponían y requirió mucho menos ajuste. Ambos resultados describen el escenario del proyecto (datos tabulares de tamaño medio extraídos de un ERP, un solo desarrollador y un plazo acotado) y justifican adoptar XGBoost, integrado con la API de Scikit-learn (**Pedregosa2011**), como algoritmo para los tres casos de uso.

2.2.4 FEATURE ENGINEERING

La ingeniería de características (*feature engineering*) es el conjunto de operaciones que transforman los datos crudos en las variables predictoras que recibe el modelo. **Kuhn2019<empty citation>** la sitúan dentro del proceso de modelado predictivo, junto con la división de datos, el remuestreo, el ajuste de hiperparámetros y la evaluación, y sostienen que la representación de los predictores condiciona el desempeño tanto como la elección del algoritmo. **Geron2022<empty citation>** muestra el mismo proceso desde la práctica con Scikit-learn: limpieza, codificación de variables categóricas, escalado y encadenamiento de esas transformaciones en *pipelines* que se ajustan únicamente con datos de entrenamiento, para evitar que información del conjunto de prueba se filtre al modelo.

Tres grupos de técnicas son relevantes para este proyecto. El primero es la codificación de predictores categóricos, como el cliente, el tipo de proyecto o el rubro: **Kuhn2019<empty citation>** describen la codificación *one-hot* o por variables indicadoras, el agrupamiento de categorías poco frecuentes y las codificaciones supervisadas, que reemplazan cada categoría por un estadístico de la respuesta, útiles cuando la variable tiene muchos niveles. El segundo es la transformación de predictores numéricos, que incluye el centrado y escalado, las transformaciones para corregir asimetría y la discretización en intervalos. El tercero, y el más sensible en un histórico operativo, es el manejo de datos faltantes: los mismos autores recomiendan visualizar primero el patrón de ausencia, y luego decidir entre eliminar observaciones o variables, imputar los valores o emplear modelos tolerantes a faltantes. Esta última opción se conecta con los defectos de calidad del ERP identificados en el análisis de datos y con la capacidad de XGBoost mencionada en la subsección anterior.

La selección de variables completa el proceso. **Guyon2003<empty citation>** le atribuyen tres objetivos: mejorar el desempeño del predictor, obtener modelos más rápidos y económicos, y comprender mejor el proceso que generó los datos. **Kuhn2019<empty citation>** clasifican los métodos en filtros, que evalúan cada variable de forma independiente del modelo, y envoltentes (*wrappers*), que buscan subconjuntos de variables evaluando el modelo sobre cada candidato mediante estrategias voraces, como la eliminación recursiva, o globales; y advierten que, si la selección se realiza fuera del esquema de remuestreo, la estimación del error queda sesgada de

forma optimista.

Como decisión de diseño del proyecto, la ingeniería de características se implementa en la capa Gold del pipeline de datos con Polars, y construye para cada proyecto un vector de variables a partir de sus atributos maestros y de agregados calculados sobre sus registros transaccionales a una fecha de corte, como totales y proporciones acumuladas. Un requisito propio de este caso es que las mismas variables deben poder calcularse tanto desde el histórico MySQL del portal anterior, que alimenta el entrenamiento, como desde el ERP nuevo en PostgreSQL, sobre el que se hará la inferencia; sin ese mapeo común, un modelo entrenado con una fuente no puede aplicarse a la otra.

2.3 INGENIERÍA DE DATOS

2.3.1 PROCESOS ETL Y ELT

2.3.2 ARQUITECTURA MEDALLION (BRONZE, SILVER, GOLD)

2.3.3 CALIDAD DE DATOS

2.4 ANALÍTICA PREDICTIVA

2.4.1 DEFINICIÓN Y APLICACIONES

2.4.2 ANALÍTICA PREDICTIVA EN GESTIÓN DE PROYECTOS

2.4.3 PREDICCIÓN DE DESVIACIONES Y COSTOS

2.5 GESTIÓN DE PROYECTOS EPC

2.5.1 DEFINICIÓN Y CARACTERÍSTICAS

2.5.2 CERTIFICACIONES Y CONTROL DE AVANCE

2.5.3 DESVIACIONES PRESUPUESTARIAS Y DE CRONOGRAMA

2.6 INGENIERÍA DE SOFTWARE

2.6.1 PROCESO DE DESARROLLO DE SOFTWARE

2.6.2 METODOLOGÍA DE DESARROLLO ITERATIVA E INCREMENTAL

2.6.3 ARQUITECTURA DE SOFTWARE POR CAPAS

2.7 TECNOLOGÍAS DEL STACK DE DESARROLLO

2.7.1 PYTHON Y SU ECOSISTEMA ML

2.7.2 DUCKDB Y POLARS

2.7.3 FASTAPI COMO MICROSERVICIO

2.7.4 NESTJS Y ANGULAR

2.8 GESTIÓN DE IDENTIDAD Y ACCESO

2.8.1 OAUTH 2.0 Y OPENID CONNECT

2.8.2 KEYCLOAK

2.9 MODEL CONTEXT PROTOCOL (MCP)

2.9.1 DEFINICIÓN Y ARQUITECTURA

2.9.2 INTEGRACIÓN DE SISTEMAS EMPRESARIALES CON MODELOS DE LENGUAJE

2.10 INTELIGENCIA DE NEGOCIOS

2.10.1 DEFINICIÓN Y REPORTE DESCRIPTIVOS

2.10.2 *POWER BI*

CAPÍTULO III

ANÁLISIS Y DISEÑO DEL SISTEMA PROPUESTO

3.1 ANÁLISIS DE DATOS HISTÓRICOS

3.1.1 EXPLORACIÓN Y CARACTERIZACIÓN DE LAS FUENTES DE DATOS

3.1.2 IDENTIFICACIÓN DE VARIABLES PREDICTORAS

3.1.3 DEFINICIÓN DE CASOS DE USO PREDICTIVOS

3.2 ANÁLISIS DE PROCESOS DEL PORTAL ANTERIOR

3.2.1 RELEVAMIENTO DE PROCESOS DEL PORTAL ANTERIOR

3.2.2 PROCESOS A REIMPLEMENTAR EN EL ERP

3.2.3 DEFECTOS Y BRECHAS QUE AFECTAN LA CALIDAD DE LOS DATOS

3.3 REQUERIMIENTOS DEL SISTEMA

3.3.1 REQUERIMIENTOS FUNCIONALES

3.3.2 REQUERIMIENTOS NO FUNCIONALES

3.4 DISEÑO DE LA ARQUITECTURA

3.4.1 ARQUITECTURA POR CAPAS

3.4.2 DISEÑO DEL PIPELINE DE DATOS

3.4.3 DISEÑO DE MODELOS ML

3.4.4 DISEÑO DE LA API PREDICTIVA

3.4.5 DISEÑO DEL DASHBOARD

3.4.6 DISEÑO DE LA AUTENTICACIÓN CENTRALIZADA

3.4.7 DISEÑO DE LAS INTEGRACIONES CON MCP Y POWER BI

CAPÍTULO IV

CONSTRUCCIÓN E IMPLEMENTACIÓN DEL SISTEMA

4.1 INCREMENTO 1: PROCESOS DEL PORTAL ANTERIOR MIGRADOS AL ERP

4.2 INCREMENTO 2: AUTENTICACIÓN CENTRALIZADA CON KEYCLOAK

4.3 INCREMENTO 3: PIPELINE DE DATOS Y ENTRENAMIENTO DE MODELOS

4.3.1 INTEGRACIÓN DUCKDB CON MYSQL Y POSTGRESQL

4.3.2 TRANSFORMACIÓN DE DATOS CON POLARS Y FEATURE ENGINEERING

4.3.3 MODELO DE DESVIACIÓN EN CERTIFICACIONES

4.3.4 MODELO DE SOBRECOSTO DE PROYECTOS

4.3.5 MODELO DE RETRASO EN LA FECHA DE CIERRE

4.4 INCREMENTO 4: MICROSERVICIO FASTAPI E INTEGRACIÓN CON EL ERP

4.4.1 DESARROLLO DEL MICROSERVICIO FASTAPI

4.4.2 ENDPOINTS DE PREDICCIÓN EN NESTJS

4.4.3 DASHBOARD DE INTELIGENCIA PREDICTIVA EN ANGULAR

4.5 INCREMENTO 5: INTEGRACIÓN CON POWER BI Y MCP

4.5.1 API DE DATOS PARA POWER BI

4.5.2 SERVIDOR MCP

4.6 EVALUACIÓN Y VALIDACIÓN

4.6.1 MÉTRICAS DE RENDIMIENTO DE MODELOS

4.6.2 PRUEBAS CON USUARIOS CLAVE

CONCLUSIONES

RECOMENDACIONES

REFERENCIAS BIBLIOGRÁFICAS